

AI & RAG Guide — Vermont Health Platform (HTR)

Audience: AI/ML engineers, backend developers. **Version:** 4.3.0 **Stack:** LlamaIndex · Groq (Llama 3.3-70b) · Anthropic (Claude Sonnet 4.6) · OpenAI Embeddings · FlashRank · pgvector · pdfplumber

Table of Contents

- 1. Overview
- 2. Architecture
- 3. Document Ingestion Pipeline
- 4. Medicaid Eligibility RAG Subsystem
- 5. Embedding Model
- 6. Vector Store (pgvector)
- 7. Hybrid Retrieval
- 8. Re-ranking
- 9. LLM Routing
- 10. Chat API
- 11. Personalized Learning Generation
- 12. Extending the Pipeline
- 13. Monitoring & Debugging

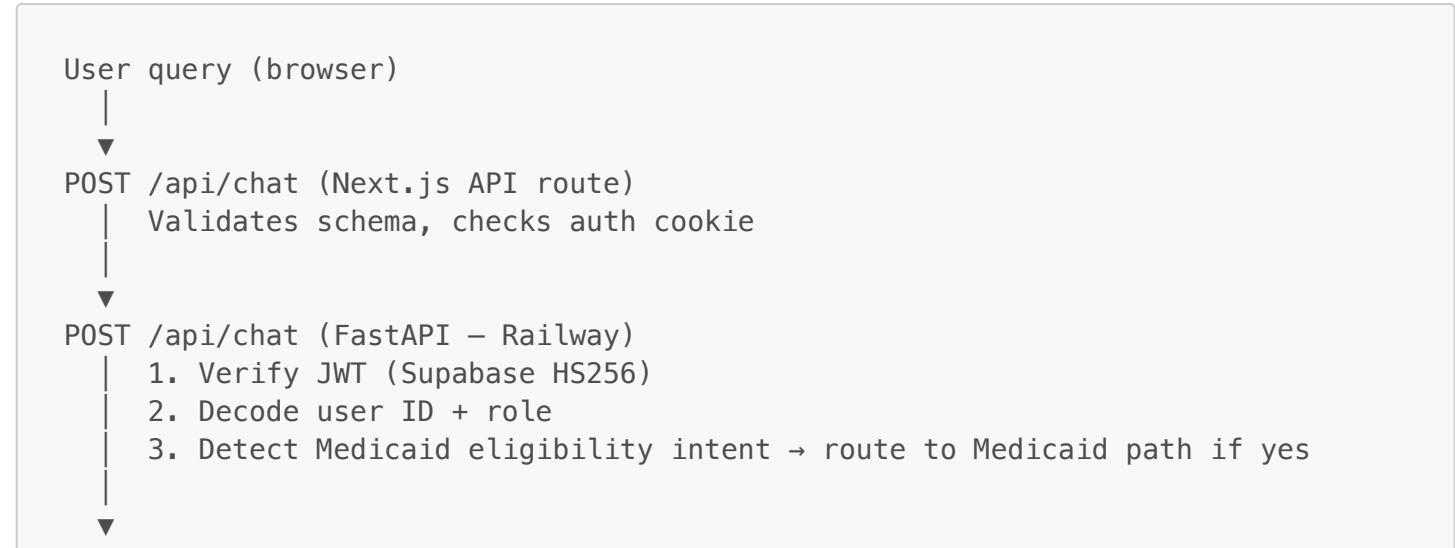
1. Overview

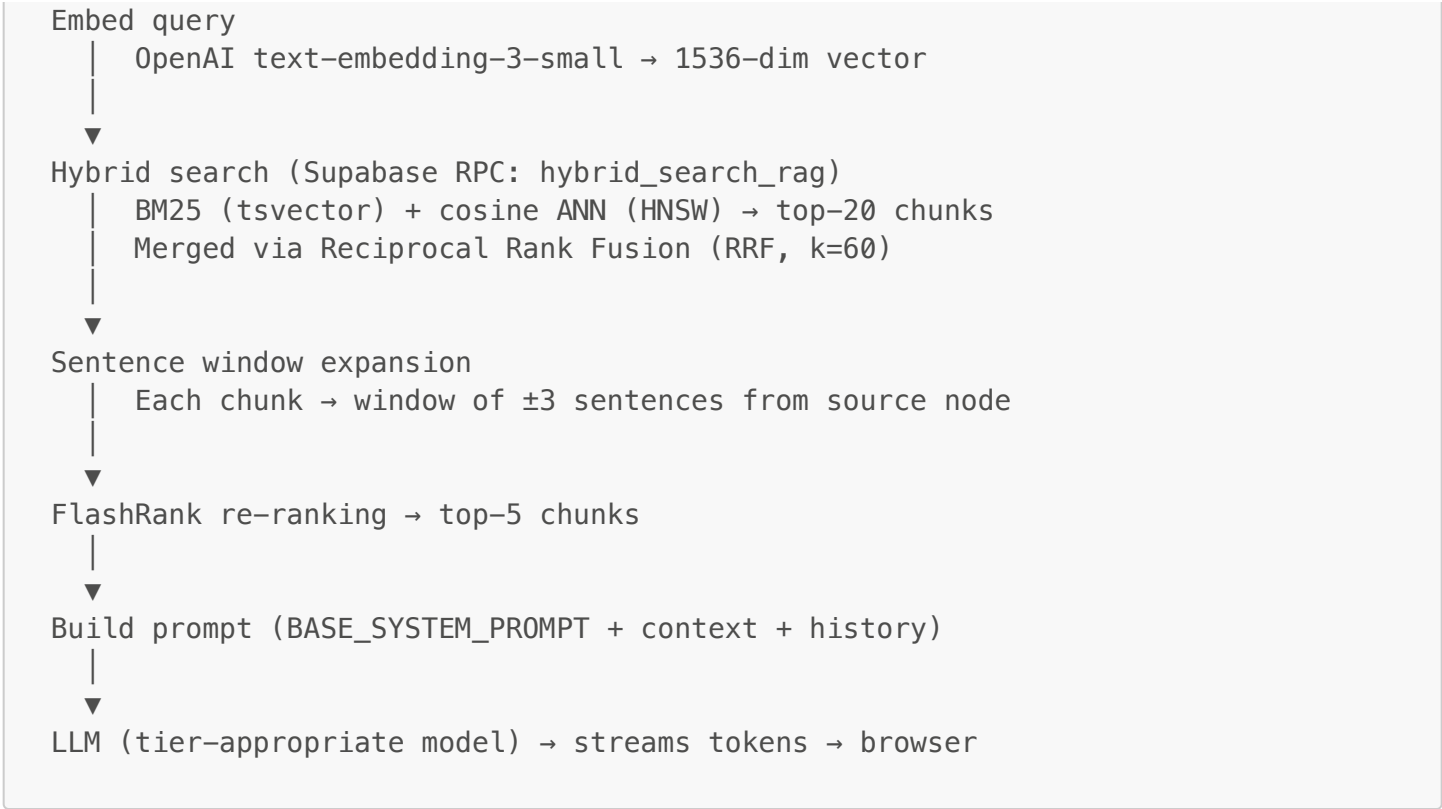
The HTR AI Analyst is a Retrieval-Augmented Generation (RAG) system that answers questions about U.S. healthcare transformation and Vermont Medicaid eligibility by grounding responses in a curated knowledge base. It combines:

- **Hybrid retrieval** — BM25 keyword search merged with dense vector similarity via Reciprocal Rank Fusion (RRF)
- **Cross-encoder re-ranking** — FlashRank (~~ms-marco-MiniLM-L-12-v2~~) re-scores retrieved chunks before passing to the LLM
- **Role-based LLM routing** — Free/Student users get Groq Llama 3.1-8b; Subscriber/Professional users get Llama 3.3-70b; Advisory/Admin users get Claude Sonnet 4.6
- **Dual ingestion pipeline** — General content uses sentence-window chunking; Medicaid eligibility documents use a specialized section-aware parser
- **Medicaid intent detection** — Queries about Vermont Medicaid eligibility are automatically routed to a scoped retrieval path backed by 23 official Vermont state documents
- **Streaming responses** — All chat responses stream token-by-token back to the browser

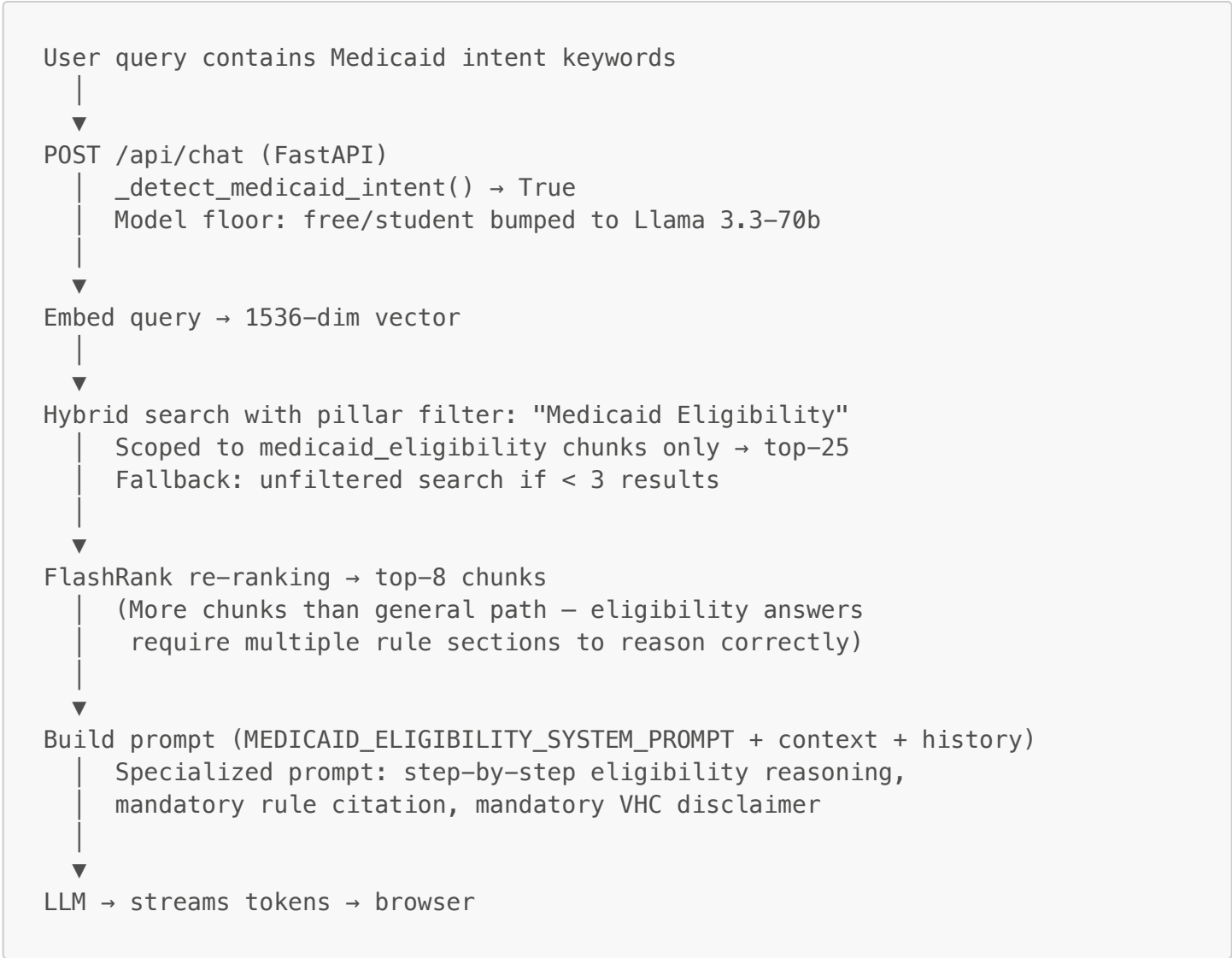
2. Architecture

General Query Path





Medicaid Eligibility Query Path



3. Document Ingestion Pipeline

Source Documents

The knowledge base is built from three source types:

Source	Location	Content	Chunking Strategy
--------	----------	---------	-------------------

Source	Location	Content	Chunking Strategy
Sanity CMS	—	Policy analyses, blog posts, academy modules, case studies, definitions	SentenceWindowNodeParser (±3 sentences)
General PDFs	backend/data/*.pdf	Technical reports, white papers, government documents	SentenceWindowNodeParser (±3 sentences)
Medicaid Eligibility PDFs	backend/data/medicaid_eligibility/*.pdf	Vermont Medicaid rules, income charts, federal regulations	Section-aware chunking (see Section 4)

Dual Pipeline in `build_index()`

The ingestion pipeline has two separate node paths. Medicaid documents **must not** be processed by `SentenceWindowNodeParser` — that parser splits content at sentence boundaries, which destroys the conditional if/then logic in regulatory text (e.g., "IF age ≥ 65 AND income ≤ X THEN eligible for MABD" split across 3 chunks loses the eligibility logic). Instead, Medicaid docs go directly to `TextNode` objects after section-aware chunking.

```
async def build_index():
    # — Path A: Medicaid eligibility docs (specialized parser) —————
    medicaid_nodes = []
    if MEDICAID_DIR.exists():
        medicaid_docs = parse_medicaid_directory(MEDICAID_DIR)
        for doc in medicaid_docs:
            medicaid_nodes.append(TextNode(
                text=doc.text,
                metadata=doc.metadata,    # includes source_type, program_type,
part, section_heading
            ))

    # — Path B: General PDFs + Sanity CMS (sentence window) —————
    general_documents = []
    # ... load PDFs with SimpleDirectoryReader + fetch Sanity content ...

    parser = SentenceWindowNodeParser.from_defaults(
        window_size=3,
        window_metadata_key="window",
        original_text_metadata_key="original_text",
    )
    general_nodes = parser.get_nodes_from_documents(general_documents)

    # — Safety pass (applied to ALL nodes before embedding) —————
    # PostgreSQL rejects NUL bytes; OpenAI embeddings reject inputs > 8192
tokens.
    for n in all_nodes:
        n.text = n.text.replace("\x00", "")                # strip NUL bytes
```

```
        if len(n.text) > 30_000:
            n.text = n.text[:30_000] # ~7500 tokens, safely
under limit

# — Embed + store —————
all_nodes = medicaid_nodes + general_nodes
index = VectorStoreIndex(all_nodes, storage_context=storage_context)
return index
```

Sanity GROQ Queries

The following content types are indexed from Sanity (defined in `backend/services/indexing.py`):

```
SANITY_QUERIES = {
  "policyAnalysis": """*[_type=="policyAnalysis" && defined(slug.current)]{
    _id, title, pillar, summary,
    "bodyText": pt::text(body)
  }""",
  "post": """*[_type=="post" && defined(slug.current)]{
    _id, title, "bodyText": pt::text(body)
  }""",
  "academyModule": """*[_type=="academyModule" && defined(slug.current)]{
    _id, title, pillar, summary, learningObjectives,
    "bodyText": pt::text(body)
  }""",
  "caseStudy": """*[_type=="caseStudy" && defined(slug.current)]{
    _id, title, pillar, summary, "bodyText": pt::text(body)
  }""",
  "definition": """*[_type=="definition"]{
    _id, term, description, pillars
  }""",
  "analystNote": """*[_type=="analystNote"]{
    _id, title, pillar, "bodyText": pt::text(body)
  }""",
  "webinar": """*[_type=="webinar" && defined(slug.current)]{
    _id, title, pillar, description
  }""",
  "report": """*[_type=="report" && defined(slug.current)]{
    _id, title, pillar, abstract
  }""",
}
```

Triggering Re-ingestion

```
# Manual trigger
curl -X POST https://your-backend.railway.app/api/ingest \
  -H "Authorization: Bearer YOUR_INGEST_SECRET"

# Check status
curl https://your-backend.railway.app/api/ingest/status \
  -H "Authorization: Bearer YOUR_INGEST_SECRET"
```

Ingestion is also triggered automatically by Sanity webhooks on content publish. The admin dashboard `/admin/ingest` has a "Trigger Ingest" button that calls the same endpoint.

4. Medicaid Eligibility RAG Subsystem

Why a Specialized Subsystem

Vermont Medicaid eligibility documents have structural properties that defeat standard PDF parsers and sentence-window chunkers:

Problem	Standard RAG failure	Our solution
Tables (income charts, FPL thresholds)	<code>pypdf</code> outputs garbled text with misaligned columns	<code>pdfplumber</code> extracts tables as structured markdown
Conditional logic ("IF age AND income THEN eligible")	Sentence-window splits conditions across chunks	Section-aware chunking keeps rules intact
Multi-column layouts	<code>pypdf</code> reads columns in wrong order	<code>pdfplumber</code> respects reading order
Regulatory hierarchy (Part → Section → Subsection)	Lost in flat sentence chunks	Section headings detected and preserved as metadata
Cross-references ("see Part 3, Section 4.2")	No context to resolve them	The combined HBEE PDF is indexed as a single source

Document Collection

23 PDF documents are stored in `backend/data/medicaid_eligibility/`. The collection was assembled from official Vermont state government sources and the GPO (Government Printing Office) for federal rules.

File	Source	Content
<code>HBEE-Combined-All-Parts-Dec2025.pdf</code>	humanservices.vermont.gov	All 8 HBEE parts combined (Dec 2025) — primary reference
<code>HBEE-Part-1-General-Provisions-and-Definitions.pdf</code>	humanservices.vermont.gov	Definitions, general provisions
<code>HBEE-Part-2-Eligibility-Standards.pdf</code>	humanservices.vermont.gov	Eligibility groups (Medicaid, Dr. Dynasaur, VHAP, etc.)
<code>HBEE-Part-3-Nonfinancial-Eligibility-Requirements.pdf</code>	humanservices.vermont.gov	Residency, citizenship, immigration status
<code>HBEE-Part-4-Special-Rules-for-Medicaid-LTC.pdf</code>	humanservices.vermont.gov	Long-term care Medicaid rules
<code>HBEE-Part-5-Financial-Methodologies.pdf</code>	humanservices.vermont.gov	Income/asset calculation rules
<code>HBEE-Part-6-Adopted-Rule-16-099.pdf</code>	humanservices.vermont.gov	2016 rule adoption
<code>HBEE-Part-7-Eligibility-and-Enrollment-Procedures.pdf</code>	humanservices.vermont.gov	Application & enrollment process
<code>HBEE-Part-8-Adopted-Scrubbed.pdf</code>	humanservices.vermont.gov	Most recent amendments
<code>MABD-PIL-Income-Chart-2026.pdf</code>	dvha.vermont.gov	MABD Protected Income Level chart (2026)
<code>MAGI-Income-Methodology.pdf</code>	dvha.vermont.gov	MAGI calculation methodology
<code>Standards-Change-Healthcare-2026.pdf</code>	dvha.vermont.gov	2026 income standard changes
<code>2026-MCA-FPL-Chart.pdf</code>	healthconnect.vermont.gov	FPL chart for Medicaid/CHIP (2026)
<code>2026-MCA-PIL-FPL-Disregard-Chart.pdf</code>	healthconnect.vermont.gov	PIL/FPL disregard chart (2026)

File	Source	Content
2026-Eligibility-Income-Thresholds-QHP.pdf	healthconnect.vermont.gov	QHP subsidy income thresholds (2026)
Health-Program-Eligibility-Tables.pdf	dvha.vermont.gov	All programs at a glance
VPharm-Income-Guidelines-2026.pdf	dvha.vermont.gov	Vermont pharmacy program income limits
MABD-NewApplicants-Attestation-Verification.pdf	dvha.vermont.gov	Required verification documents
Vermont-Medicaid-General-Provider-Manual.pdf	vtmedicaid.com	Provider coverage and billing rules
Choices-for-Care-Regulations.pdf	asd.vermont.gov	Full long-term care Medicaid regulations
Choices-for-Care-Eligibility-At-A-Glance.pdf	asd.vermont.gov	LTC eligibility quick reference
Choices-for-Care-Options-At-A-Glance.pdf	asd.vermont.gov	LTC program options summary
42-CFR-Part-435-Medicaid-Eligibility.pdf	govinfo.gov (GPO)	Federal CFR Title 42 Part 435

Total: 23 PDFs → **1,246 section chunks** after parsing.

The Medicaid Parser ([backend/services/medicaid_parser.py](#))

The parser uses `pdfplumber` instead of `pypdf` and applies section-aware chunking.

Table extraction:

```
def _table_to_markdown(table: list) -> str:
    """Convert a pdfplumber table (list of rows) to markdown."""
    rows = [[clean(str(cell or "")).strip() for cell in row] for row in table]
    rows = [r for r in rows if any(c for c in r)]
    header = "| " + " | ".join(rows[0]) + " |"
    separator = "| " + " | ".join(["---"] * len(rows[0])) + " |"
    body = "\n".join("| " + " | ".join(r) + " |" for r in rows[1:])
    return "\n".join([header, separator, body])
```

Tables are kept as intact markdown chunks so the LLM receives structured income data (e.g., household size → income threshold) rather than garbled text.

Section heading detection:

```
_HEADING_PATTERNS = [
    re.compile(r"^(Part\s+\d+)", re.IGNORECASE),
    re.compile(r"^(Section\s+[\d.]+)", re.IGNORECASE),
    re.compile(r"^(\\d{1,2}\\.\\d{1,2}(?:\\.\\d{1,2})?)\\s+\\S"), # 2.3 or 2.3.1
    re.compile(r"^( [A-Z] [A-Z\\s]{3,50})$"), # ALL-CAPS
]
headings
    re.compile(r"^(CHAPTER\s+\d+|SUBCHAPTER\s+\\w+)", re.IGNORECASE),
]
```

When a heading is detected, the current section is flushed as a chunk and a new section begins. Oversized sections are split at paragraph boundaries (never mid-sentence).

Chunk metadata:

Every Medicaid chunk carries metadata that enables scoped retrieval and accurate citations:

```
{
  "source":      "pdf",
  "source_type": "medicaid_eligibility", # used for pillar filter
  "program_type": "HBEE Rules",          # inferred from
filename/content
  "part":        "Part 2",                # HBEE part number when
determinable
  "section_heading": "2.3 Eligibility for...", # nearest detected heading
  "page_num":      14,
  "file_name":      "HBEE-Part-2-Eligibility-Standards.pdf",
  "pillar":         "Medicaid Eligibility", # used by HybridRetriever
filter
}
```

Program type inference — automatically assigned based on filename and early-page content:

Keyword	Assigned program_type
choices, long-term, ltc	Long-Term Care / Choices for Care
magi	MAGI Medicaid
mabd	MABD Medicaid
vpharm	VPharm
provider	Provider Rules
42-cfr, cfr	Federal Rules (42 CFR 435)
fpl, threshold	Income Thresholds
hbee, health benefits	HBEE Rules

Parse results by program type:

Program Type	Chunks
HBEE Rules	974
MAGI Medicaid	118
Long-Term Care / Choices for Care	43
Federal Rules (42 CFR 435)	41
Provider Rules	28
MABD Medicaid	23
Vermont Medicaid / VPharm / Income Thresholds	19
Total	1,246

Medicaid Intent Detection (routers/chat.py)

The chat endpoint detects Medicaid eligibility intent before retrieval using a two-pass check:

```
_MEDICAID_KEYWORDS = {
  # Program names
  "medicaid", "dr. dinosaur", "vhap", "mabd", "magi",
  "choices for care", "vpharm", "vermont health connect",
  # Eligibility concepts
```

```

    "eligible", "eligibility", "qualify", "enroll", "enrollment",
    "health insurance", "health coverage", "health benefits",
    # Financial
    "income limit", "income threshold", "fpl", "federal poverty",
    "protected income", "pil chart", "household size",
    # Program-specific
    "long-term care", "nursing home", "ltc medicaid", "spend down",
    # Vermont-specific
    "hbee", "dvha", "affordable care act",
    ...
}

_MEDICAID_QUESTION_PHRASES = (
    "am i eligible", "do i qualify", "can i get", "how do i apply",
    "what are the requirements", "who qualifies", "will i qualify", ...
)

def _detect_medicaid_intent(message: str) -> bool:
    lower = message.lower()
    if any(phrase in lower for phrase in _MEDICAID_QUESTION_PHRASES):
        return True
    return any(kw in lower for kw in _MEDICAID_KEYWORDS)
```

When `_detect_medicaid_intent()` returns `True`:

- 1. Retrieval is scoped to `pillar="Medicaid Eligibility"` with `top_k=25`
- 2. Re-ranking returns `top_k=8` (vs. 5 for general queries)
- 3. System prompt switches to `MEDICAID_ELIGIBILITY_SYSTEM_PROMPT`
- 4. Free/student users are bumped to Llama 3.3-70b (eligibility reasoning requires the larger model)

Medicaid System Prompt

```

MEDICAID_ELIGIBILITY_SYSTEM_PROMPT = (
    "You are a Vermont Medicaid eligibility specialist. "
    "Your role is to help Vermont residents understand whether they may qualify "
    "for "
    "Medicaid or related health coverage programs based on official Vermont "
    "state rules. "
    ...
    "When answering eligibility questions:\n"
    "1. Cite the specific rule part and section (e.g. 'HBEE Part 2, Section "
    "2.3').\n"
    "2. Walk through eligibility criteria step by step – category, residency, "
    "income, and program-specific requirements.\n"
    "3. Use the 2026 income charts to give specific dollar thresholds by "
    "household size.\n"
    "4. Ask clarifying questions if the user's situation is ambiguous.\n"
    "5. Always close with: 'This is general information based on Vermont state "
    "rules. "
    "For a formal eligibility determination, apply at Vermont Health Connect "
    "(healthconnect.vermont.gov) or call 1-800-250-8427.'\n"
    "6. Never fabricate rule sections or income numbers."
)
```

The mandatory Vermont Health Connect disclaimer (item 5) is required on every eligibility response. The system prompt enforces it — it is not optional and must not be removed.

Re-ingesting Medicaid Documents

To add a new Medicaid document:

- 1. Place the PDF in `backend/data/medicaid_eligibility/`

2. Trigger a full re-ingest: `POST /api/ingest`

The parser will automatically detect the program type from the filename and content. To update an existing document, replace the file and re-ingest.

5. Embedding Model

Setting	Value
Provider	OpenAI
Model	<code>text-embedding-3-small</code>
Dimensions	1536
Max tokens	8,192
Cost	~\$0.02 / million tokens

Configured in `backend/services/llm.py`:

```
from llama_index.embeddings.openai import OpenAIEmbedding

Settings.embed_model = OpenAIEmbedding(
    model="text-embedding-3-small",
    api_key=OPENAI_API_KEY,
)
```

Note: The pgvector table is configured with `embed_dim=1536`. If you change the embedding model, you must drop and recreate the `rag_documents` table with the correct dimension.

The same model is used for both ingestion and retrieval, ensuring vector space consistency.

6. Vector Store (pgvector)

Embeddings are stored in Supabase PostgreSQL using the `pgvector` extension. The `rag_documents` table holds chunked text alongside 1536-dimensional embedding vectors.

Storage Configuration

```
from llama_index.vector_stores.postgres import PGVectorStore

vector_store = PGVectorStore.from_params(
    connection_string=SUPABASE_DB_URL,
    async_connection_string=SUPABASE_DB_URL.replace("postgresql://",
"postgresql+asyncpg://"),
    table_name="rag_documents",
    embed_dim=1536,
)
```

HNSW Index

```
CREATE INDEX idx_rag_embedding
ON public.rag_documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

- `m = 16` — graph branching factor. Higher values increase recall but slow index construction.

- `ef_construction = 64` — search width during build.
- `ef_search` — set at query time; controls accuracy vs. latency tradeoff.

Fallback — Local JSON Storage

If `SUPABASE_DB_URL` is not set, the index falls back to LlamaIndex's local JSON vector store in `backend/storage/`. Suitable for development without Supabase.

7. Hybrid Retrieval

The `HybridRetriever` class in `backend/services/retrieval.py` calls the `hybrid_search_rag` Supabase RPC.

Dense Retrieval (Vector)

Cosine similarity between query embedding and stored document embeddings via HNSW index. Good at semantic similarity — finds related content even when exact keywords differ.

Sparse Retrieval (BM25)

PostgreSQL full-text search: `to_tsvector` for indexing, `plainto_tsquery` for parsing, `ts_rank` for scoring. Good at exact keyword matching — required for proper nouns, acronyms, and legal terms (e.g., "MABD", "PIL chart", "HBEE Part 5").

Reciprocal Rank Fusion

$$\text{RRF score} = (\text{dense_weight} / (60 + \text{dense_rank})) + (\text{sparse_weight} / (60 + \text{sparse_rank}))$$

Default weights: `dense=0.6`, `sparse=0.4`.

Retrieval Parameters by Query Type

Parameter	General queries	Medicaid eligibility queries
<code>top_k</code> (before rerank)	20	25
Final chunks (after rerank)	5	8
Pillar filter	From request (optional)	" <code>Medicaid Eligibility</code> " (forced)
Sentence window expansion	Yes (± 3 sentences)	No (section chunks used as-is)
Fallback if < 3 results	Unfiltered search	Unfiltered search

The higher `top_k` and final chunk count for Medicaid queries is intentional: an eligibility determination typically requires combining income thresholds (from one chunk), residency rules (another), program category definitions (another), and procedural requirements (another). Passing only 5 chunks to the LLM risks omitting a critical rule.

8. Re-ranking

After hybrid retrieval, FlashRank applies cross-encoder re-ranking to select the top chunks.

FlashRank Configuration

```
from flashrank import Ranker

_ranker = Ranker(
    model_name="ms-marco-MiniLM-L-12-v2",
```

```
        cache_dir="/tmp/flashrank"
    )
```

Model: `ms-marco-MiniLM-L-12-v2` — a cross-encoder fine-tuned on MS MARCO passage ranking. Scores (query, passage) pairs directly rather than comparing embeddings, giving significantly better ranking quality than bi-encoder retrieval alone.

The model downloads to `/tmp/flashrank` on first use (~100MB). Subsequent startups reuse the cached model.

9. LLM Routing

Base Routing (all queries)

Role	Model	Notes
<code>free</code> / <code>student</code>	Groq Llama 3.1-8b-instant	Fastest, lower quality
<code>subscriber</code> / <code>professional</code>	Groq Llama 3.3-70b-versatile	High quality, low latency
<code>advisory</code> / <code>admin</code>	Claude Sonnet 4.6 (Anthropic)	Highest quality, longest context

Medicaid Eligibility Override

When `_detect_medicaid_intent()` is `True`, free/student users are promoted to Llama 3.3-70b for that query only. Advisory/admin users remain on Claude Sonnet 4.6.

```
def _get_llm_for_medicaid(user: AuthedUser):
    if user.role in ("free", "student"):
        # Bump up – eligibility reasoning requires the larger model
        return FallbackLLM(
            primary=GroqLLM(model=MODEL_SUBSCRIBER),
            fallbacks=[GroqLLM(model=MODEL_FREE)],
        )
    return get_llm_for_role(user.role)
```

Fallback Chain

Each model is wrapped in `FallbackLLM`, which catches retryable errors (429, 5xx, rate limit) and tries the next model in the chain:

- Advisory/Admin: Claude Sonnet 4.6 → Llama 3.3-70b → Llama 3.1-8b → GPT-4o-mini
- Subscriber: Llama 3.3-70b → Llama 3.1-8b → GPT-4o-mini
- Free/Student: Llama 3.1-8b → GPT-4o-mini

Dev Bypass

When `SUPABASE_JWT_SECRET` is not set, all requests are treated as `role="subscriber"` (Llama 3.3-70b). This allows local development without a valid JWT.

10. Chat API

Endpoint

```
POST /api/chat
Auth: Bearer JWT (Supabase user token)
Rate: 30/min per IP (slowapi)
```

Request Schema

```
class ChatRequest(BaseModel):
    message: str # max 2000 chars
    history: list[dict] # [{role, text}], max 4000 chars/item
    temperature: float = 0.7 # 0.0–1.0
    systemPrompt: str | None # max 800 chars; overrides default system
prompt
    conversation_id: str | None # for conversation persistence
    pillar: str | None #
policy|economics|technology|clinical|equity
```

Response

text/plain stream. Citations are appended after the response text as a sentinel:

```
[CITATIONS] [{"title": "...", "url": "...", "pillar": "...", "source_type": "..."}, ...]
[/CITATIONS]
```

The frontend strips the sentinel and renders citations separately below the response.

System Prompts

Three system prompts are in use:

BASE_SYSTEM_PROMPT — used for general healthcare policy questions (subscriber/professional role without Medicaid intent):

```
You are an expert AI Analyst for the Health Transformation Review (HTR).
Your audience consists of healthcare executives, policy makers, and economists.
Answer questions thoroughly and professionally, citing specific policies, data,
and source documents where relevant...
```

ADVISORY_SYSTEM_PROMPT — **BASE_SYSTEM_PROMPT** extended with deeper strategic analysis instructions for advisory/admin users without Medicaid intent.

MEDICAID_ELIGIBILITY_SYSTEM_PROMPT — replaces the base prompt entirely for any query with Medicaid intent. See Section 4 for the full prompt text.

Query Rewriting

For follow-up questions, a fast LLM (Llama 3.1-8b) rewrites the question into a standalone retrieval query incorporating conversation history, before the query is embedded and sent to the vector store:

```
async def _rewrite_query(message: str, history: list) -> str:
    # Uses the last 6 messages as context
    # Returns a self-contained query string optimized for retrieval
    ...
```

Falls back to the raw message if rewriting fails or there is no prior history.

Structured Output Detection

The chat endpoint detects comparison and list requests and appends format instructions to the system prompt:

- Comparison phrases ("compare", "versus", "difference between") → inject markdown table format instruction

- List phrases ("list", "summarize into", "step-by-step") → inject bullet/header format instruction

Follow-up Suggestions

```
POST /api/suggest
Auth: optional
Rate: 60/min per IP
Body: { "message": "...", "history": [...] }
Response: { "suggestions": ["string", "string", "string"] }
```

Returns 3 contextual follow-up question suggestions displayed below each AI response.

11. Personalized Learning Generation

The Personalized Learning feature (`/academy/personalized-learning`) generates a multi-week structured learning curriculum using a direct LLM call — it does not use the RAG pipeline.

Endpoint

```
POST /api/personalized-learning/generate
Auth: Bearer JWT
```

Request Schema

```
class LearningPathRequest(BaseModel):
    role: str
    topics: list[str]
    difficulty: Literal["foundational", "intermediate", "advanced"]
    weekly_hours: str # e.g. "3-5"
    goals: str
    format_preferences: list[str] = []
```

Generation Strategy

The backend constructs a detailed system prompt instructing the LLM to return a JSON curriculum with structured weeks, items, and knowledge checks. The curriculum is stored in `user_learning_paths`.

Text-to-Speech

```
POST /api/personalized-learning/tts
Auth: Bearer JWT
Body: { "text": "...", "voice": "alloy|echo|fable|onyx|nova|shimmer" }
Response: audio/mpeg (streaming MP3)
```

Uses OpenAI TTS API (`tts-1` model).

12. Extending the Pipeline

Adding a General PDF to the Knowledge Base

Place a PDF in `backend/data/` (not in a subdirectory). The `SimpleDirectoryReader` picks up all files in the root `data/` directory on next ingest. Re-ingest to index.

Chunks will be tagged `source_type: "general"` and processed through `SentenceWindowNodeParser`.

Adding a Medicaid Eligibility Document

Place a PDF in `backend/data/medicaid_eligibility/`. The medicaid parser will:

- 1. Auto-detect the program type from the filename
- 2. Extract tables as markdown
- 3. Chunk at section boundaries
- 4. Tag with `pillar: "Medicaid Eligibility"` for scoped retrieval

Re-ingest to index. No code changes needed for standard Vermont state government PDFs.

Adding a New Sanity Content Type

In `backend/services/indexing.py`, add a GROQ query to `SANITY_QUERIES`:

```
SANITY_QUERIES["myNewType"] = """*[_type=="myNewType" && defined(slug.current)]
{
  _id, title, pillar, summary,
  "bodyText": pt::text(body)
}"""
```

Trigger a re-ingest to index.

Changing Retrieval Parameters

For general queries, adjust in `routers/chat.py`:

```
nodes = HybridRetriever(supabase=supabase, top_k=30).retrieve(query_bundle)
nodes = rerank_nodes(retrieval_query, nodes, top_k=8)
```

For Medicaid queries, adjust the `is_medicaid_query` branch:

```
nodes = HybridRetriever(supabase=supabase, top_k=30,
filter_pillar=MEDICAID_PILLAR).retrieve(query_bundle)
nodes = rerank_nodes(retrieval_query, nodes, top_k=10)
```

Changing RRF Weights

Edit the `dense_weight` and `sparse_weight` variables inside the `hybrid_search_rag` SQL function in Supabase. Increase `sparse_weight` for legal/regulatory queries (exact term matching matters more); increase `dense_weight` for conceptual/semantic questions.

Adding an Agentic Tool

Define a tool in `backend/services/tools.py`:

```
from llama_index.core.tools import FunctionTool

def get_medicaid_income_limit(household_size: int, program: str) -> str:
    """Returns the 2026 income limit for a given household size and program."""
    ...

tool = FunctionTool.from_defaults(fn=get_medicaid_income_limit)
```

Register in `ALL_TOOLS` in `tools.py`. Tools are only available to `AGENTIC_ROLES` (professional, advisory, admin).

Updating Medicaid Intent Keywords

The keyword list in `routers/chat.py` (`_MEDICAID_KEYWORDS` and `_MEDICAID_QUESTION_PHRASES`) is a plain Python set/tuple — no model retraining needed. Add terms and restart the backend.

13. Monitoring & Debugging

Health Check

```
curl https://your-backend.railway.app/health
```

Returns:

```
{
  "status": "ok",
  "version": "4.2.0",
  "index_ready": true,
  "model_subscriber": "llama-3.3-70b-versatile",
  "embedding_model": "text-embedding-3-small",
  "vector_store": "pgvector",
  "auth_enabled": true,
  "reranker": "flashrank",
  "retrieval": "hybrid_bm25_vector_rrf",
  "chunking": "sentence_window_3"
}
```

Query Log

Every AI Analyst query is logged to `rag_query_log` in Supabase with: user ID, query text, role, model used, retrieved doc IDs and scores, response preview, latency, and `was_zero_result` flag.

Zero-result queries indicate no chunks scored above the similarity threshold. For Medicaid queries this is rare (1,246 chunks cover the full Vermont ruleset). For general queries, common causes are: topic not in knowledge base, unusual terminology, or content published but not yet re-ingested.

Debug Logging

The backend logs the Medicaid intent flag on every request:

```
Chat: user=abc role=subscriber msg_len=62 pillar=None medicaid=True
Medicaid retrieval: 23 nodes with pillar filter
```

Set `LOG_LEVEL=DEBUG` for verbose retrieval scoring output.

Common Issues

"Index not ready" — Startup build or load failed. Check Railway logs. Common causes: missing `SUPABASE_DB_URL`, pgvector not enabled, or invalid `OPENAI_API_KEY`.

Input bar missing on /chat page — Height chain broken. The AppShell wrapper for chat uses `[>main]:flex-1 [>main]:min-h-0 [>main]:flex [>main]:flex-col` to pass height through the `<main>` wrapper in `layout.tsx`. If you restructure `layout.tsx`, verify this chain is intact.

Medicaid answers not citing rule sections — The `MEDICAID_ELIGIBILITY_SYSTEM_PROMPT` instructs the LLM to cite sections. If citations are missing, check that `_detect_medicaid_intent()` is returning `True` for the query (visible in logs as `medicaid=True`). If it returns `False`, add the relevant keyword to `_MEDICAID_KEYWORDS`.

Income numbers in Medicaid answers are wrong — The 2026 income charts are in [MABD–PIL–Income–Chart–2026.pdf](#), [2026–MCA–FPL–Chart.pdf](#), and related files. Verify these parsed correctly by checking that table chunks appear in the vector store with `source_type="medicaid_eligibility"`. Re-ingest if needed.

High latency (>3s) — FlashRank downloads its model on first startup (~100MB, one-time). Check `rag_query_log.latency_ms` to identify whether the bottleneck is embedding, retrieval, re-ranking, or LLM generation.

NUL byte errors in PostgreSQL — The safety pass in `build_index()` strips NUL bytes (`\x00`) from all node text and metadata before embedding. If you see `ValueError: A string literal cannot contain NUL characters` in Railway logs, a new PDF may have introduced binary artifacts. The fix is already in place in [indexing.py](#) — re-triggering the ingest should resolve it.

JWT validation errors — Verify `SUPABASE_JWT_SECRET` in the backend `.env` exactly matches Supabase → Project Settings → API → JWT Secret (HS256).